Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION

Name	**:**	**Durgesh Vijay Bhangale**
Week	**:**	**Week 10**
Batch	**:**	**B1_Python**
Trainer	**:**	**Kunal**

Day 1 (20-4-26)

Project Title:
Smart IT Service Desk & System Monitoring Automation using Python

main.py

```
from tickets import Ticket
from utils import load_tickets, save_tickets, log_event
from monitor import monitor_system
from reports import daily_summary, monthly_trend

def menu():
    while True:
        print("\nSmart IT Service Desk")
        print("1. Create Ticket")
        print("2. View All Tickets")
        print("3. Search Ticket by ID")
        print("4. Update Ticket Status")
        print("5. Close Ticket")
        print("6. Delete Ticket")
        print("7. Monitor System")
        print("8. Daily Report")
        print("9. Monthly Report")
        print("10. Exit")

        choice = input("Choose option: ")
        tickets = load_tickets()

        try:
            if choice=="1":
                emp=input("Employee: "); dept=input("Dept: "); issue=input("Issue: "); cat=input("Category:
")
                if not issue: raise ValueError("Empty issue description")
                t=Ticket(emp,dept,issue,cat)
                tickets.append(t.__dict__); save_tickets(tickets); log_event("Ticket created"); print("✅
Ticket Created:",t.__dict__)
            elif choice=="2": [print(t) for t in tickets]
            elif choice=="3":
                tid=input("Enter ID: ")
                print(next((t for t in tickets if t['id']==tid),"Not found"))
```

```
        elif choice=="4":
            tid=input("Enter ID: ")
            for t in tickets:
                if t['id']==tid: t['status']="In Progress"; save_tickets(tickets); log_event("Ticket updated")
        elif choice=="5":
            tid=input("Enter ID: ")
            for t in tickets:
                if t['id']==tid: t['status']="Closed"; save_tickets(tickets); log_event("Ticket closed")
        elif choice=="6":
            tid=input("Enter ID: ")
            tickets=[t for t in tickets if t['id']!=tid]; save_tickets(tickets); log_event("Ticket deleted")
        elif choice=="7": monitor_system()
        elif choice=="8": daily_summary()
        elif choice=="9": monthly_trend()
        elif choice=="10": break
        else: print("Invalid input")
    except Exception as e:
        log_event(f"Error: {e}")
        print("❌ Error:",e)

if __name__=="__main__":
    menu()
```

tickets.py

```
import datetime, uuid

class Ticket:
    def __init__(self, employee, dept, issue, category):
        self.id = str(uuid.uuid4())[:8]
        self.employee = employee
        self.dept = dept
        self.issue = issue
        self.category = category
        self.priority = self.assign_priority(issue)
        self.status = "Open"
        self.created = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")

    def assign_priority(self, issue):
        mapping = {
            "Server Down": "P1",
            "Internet Down": "P2",
            "Laptop Slow": "P3",
            "Password Reset": "P4"
        }
        return mapping.get(issue, "P4")

class IncidentTicket(Ticket): pass
```

```
class ServiceRequest(Ticket): pass
class ProblemRecord(Ticket): pass
```

utils.py

```
import json, csv, os, datetime

DATA_FILE = "data/tickets.json"
LOG_FILE = "data/logs.txt"
BACKUP_FILE = "data/backup.csv"

def load_tickets():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE) as f: return json.load(f)
    return []

def save_tickets(tickets):
    with open(DATA_FILE,"w") as f: json.dump(tickets,f,indent=4)
    backup_tickets(tickets)

def log_event(event):
    with open(LOG_FILE,"a") as f:
        f.write(f"{datetime.datetime.now()} - {event}\n")

def backup_tickets(tickets):
    with open(BACKUP_FILE,"w",newline="") as f:
        writer=csv.DictWriter(f,fieldnames=tickets[0].keys() if tickets else [])
        if tickets: writer.writeheader(); writer.writerows(tickets)
```

monitor.py

```
import psutil
from tickets import Ticket
from utils import load_tickets, save_tickets, log_event

def monitor_system():
    cpu = psutil.cpu_percent()
    ram = psutil.virtual_memory().percent
    disk = psutil.disk_usage('/').percent

    if cpu > 90 or ram > 95 or disk > 90:
        tickets = load_tickets()
        t = Ticket("System","IT","High CPU/RAM/Disk","Monitoring")
        tickets.append(t.__dict__)
        save_tickets(tickets)
        log_event("Auto Ticket Raised due to system threshold breach")
```

```
    print(" ⚠️ Auto Ticket Raised:", t.__dict__)
```

reports.py

```
from utils import load_tickets
from collections import Counter
import datetime

def daily_summary():
    tickets = load_tickets()
    today = datetime.datetime.now().strftime("%Y-%m-%d")
    today_tickets = [t for t in tickets if t['created'].startswith(today)]
    print("Daily Summary Report")
    print("Total:", len(today_tickets))
    print("Open:", sum(1 for t in today_tickets if t['status']=="Open"))
    print("Closed:", sum(1 for t in today_tickets if t['status']=="Closed"))
    print("High Priority:", sum(1 for t in today_tickets if t['priority']=="P1"))
    print("SLA Breached:", sum(1 for t in today_tickets if t['status']=="SLA Breached"))

def monthly_trend():
    tickets = load_tickets()
    issues = [t['issue'] for t in tickets]
    dept = [t['dept'] for t in tickets]
    print("Monthly Trend Report")
    print("Most common issue:", Counter(issues).most_common(1))
    print("Department with most incidents:", Counter(dept).most_common(1))
```